

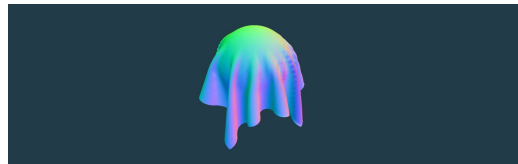
CS184/284A Spring 2026

Homework 4 Write-Up

Name: Mandy Pham

Link to webpage: <https://cal-cs184-student.github.io/hw-webpages-tpmandy24/hw4/index.html>

Link to GitHub repository: <https://github.com/cal-cs184-student/hw4-clothsim-manman-6>



Overview

For this homework, I built a full cloth simulation and shading pipeline. I started by representing cloth as a grid of point masses connected by structural, shearing, and bending springs. I then implemented physical simulation using external forces, Hooke's law, Verlet integration, and deformation constraints. After that, I added support for collisions with spheres, planes, and the cloth itself using spatial hashing for efficient self-collision handling. In the last part, I implemented several GPU shaders, including diffuse shading, Blinn-Phong shading, texture mapping, bump mapping, displacement mapping, mirror reflections, and a custom extra credit shader. One of the most interesting parts of the homework was seeing how changes in physical parameters like spring constant, density, and damping directly changed the behavior of the cloth, while shader programs completely changed how the same simulated geometry looked on screen.

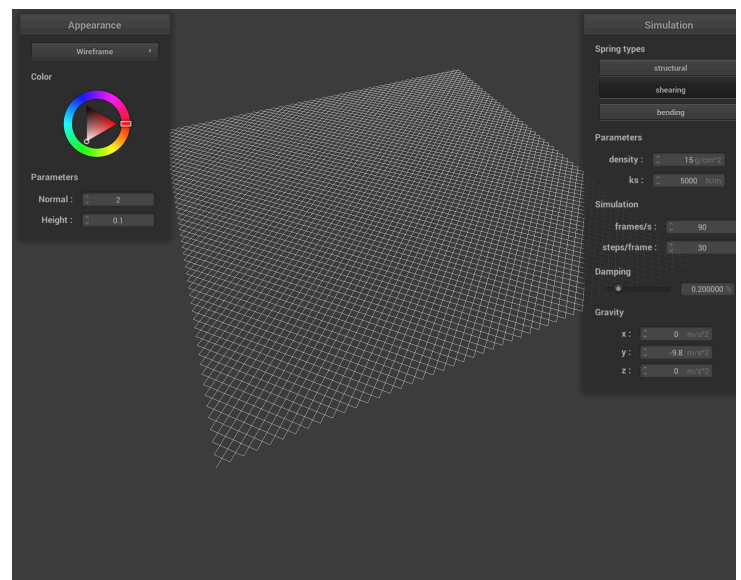
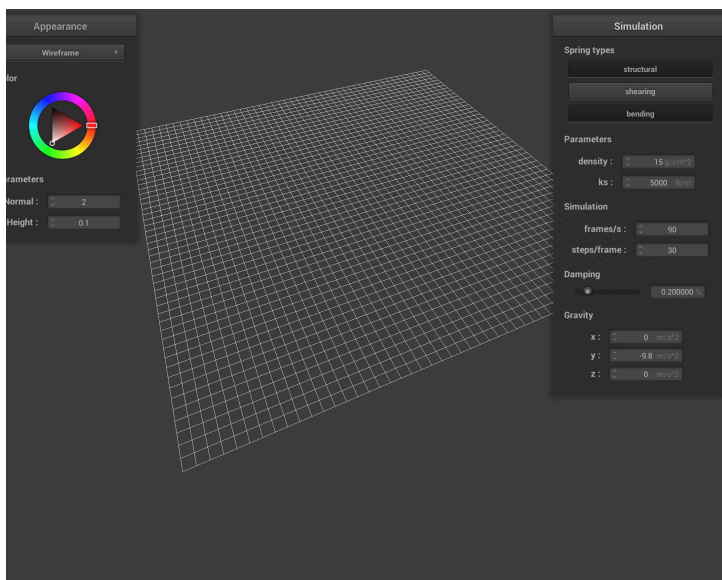
Part 1: Masses and springs

For this part, I implemented `cloth::buildGrid` to construct the cloth as a grid of point masses connected by springs. I first

created $\text{num_width_points} \times \text{num_height_points}$ point masses and stored them in `point_masses` in row-major order so that the cloth mesh would render correctly. For horizontal cloth, I varied the point mass positions over the xz -plane while keeping $y = 1$. For vertical cloth, I varied the positions over the xy -plane and assigned each point mass a small random z -offset so the cloth would not start perfectly flat. I also checked whether each grid coordinate appeared in the pinned list and marked those point masses as pinned.

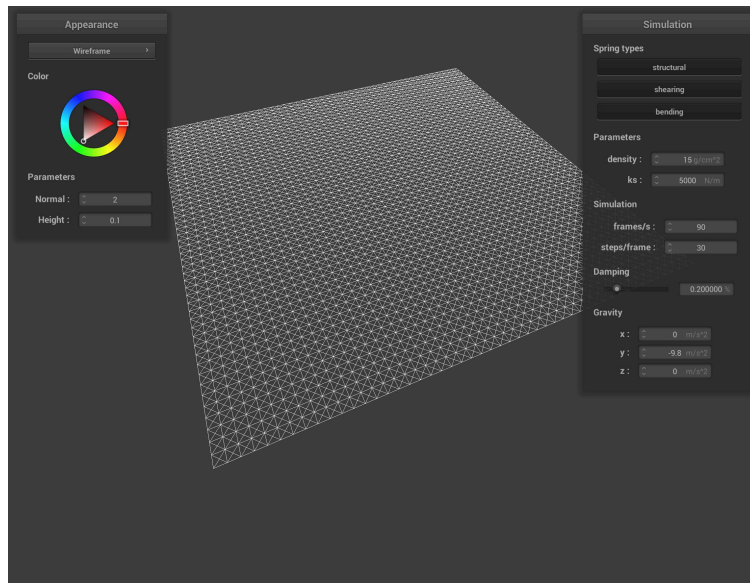
After building all point masses, I created springs between them to enforce the three required constraints. Structural springs connect each point mass to the one directly above it and directly to its left. Shearing springs connect each point mass to its upper-left and upper-right diagonal neighbors. Bending springs connect each point mass to the point mass two positions above it and two positions to its left. I made sure to handle edge cases so springs were only added when the neighbor actually existed.

To test the implementation, I loaded the pinned cloth scenes and viewed the cloth in wireframe mode. Without shearing constraints, the cloth mainly showed horizontal and vertical connections. With only shearing constraints enabled, diagonal spring connections became visible. With all constraints enabled, the cloth showed the full spring structure, including the additional longer-range bending links. This confirmed that the point masses were placed correctly and that each spring type was being added to the proper neighboring masses.



Cloth wireframe without shearing constraints.

Cloth wireframe with only shearing constraints.



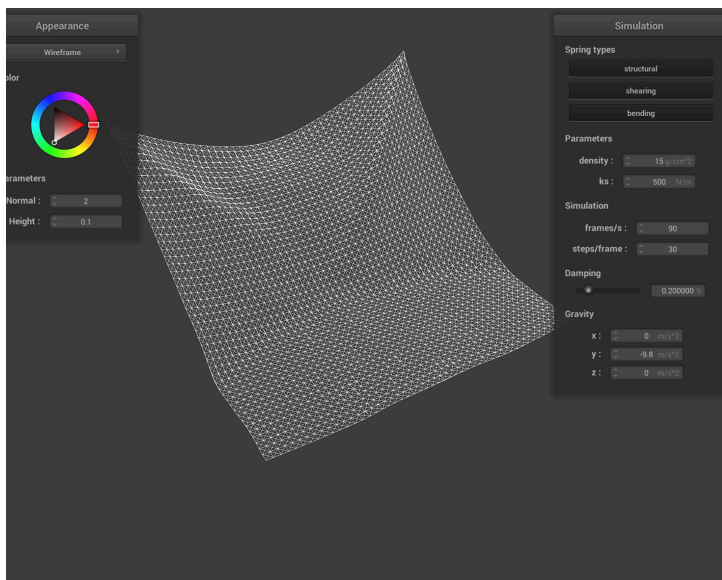
Cloth wireframe with all constraints enabled.

Part 2: Simulation via numerical integration

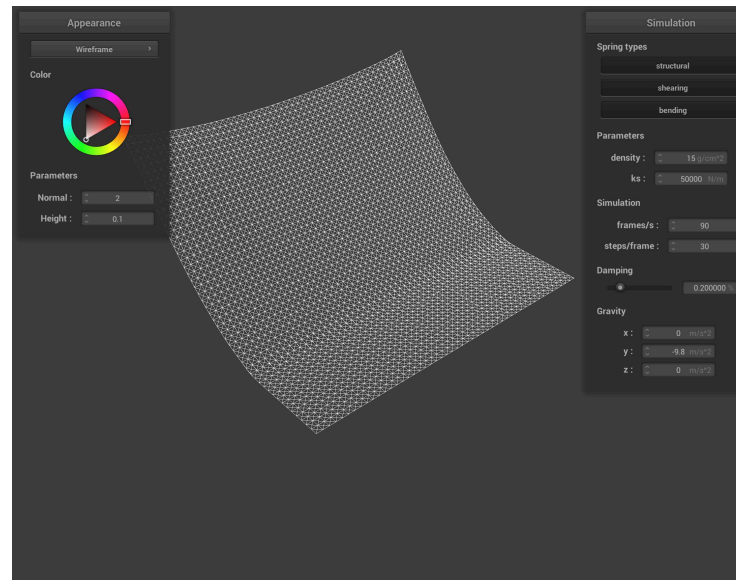
For Part 2, I implemented the simulation step in `Cloth::simulate` by computing forces on each point mass, updating positions using Verlet integration, and enforcing spring length constraints. At the start of each simulation step, I reset the force vector for every point mass and added the total external force using $F = ma$. I then computed spring forces with Hooke's law for all enabled spring types. Structural and shearing springs used the normal spring constant, while bending springs were scaled down so that bending would be weaker and the cloth could fold more naturally.

After computing all forces, I updated each unpinned point mass using damped Verlet integration. This uses the current position, previous position, acceleration, and timestep to calculate the next position while also applying damping to reduce oscillation over time. Pinned point masses were left unchanged. After the position update, I applied the Provat constraint to prevent any spring from stretching to more than 110% of its rest length in a single timestep. If a spring exceeded that limit, I corrected the positions of its endpoints, splitting the correction evenly unless one endpoint was pinned.

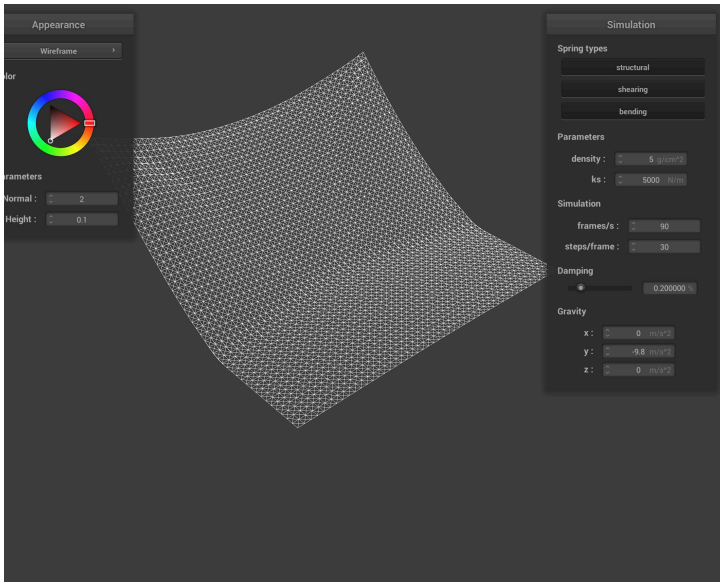
I then experimented with the simulation parameters to see how they changed the cloth's behavior. Lowering the spring constant k_s made the cloth much looser and more elastic, so it sagged more and deformed more easily as it fell. Increasing k_s made the cloth behave more stiffly and resist stretching, so it held its shape better throughout the simulation. Changing the density mainly affected how heavy the cloth felt under gravity: a low density produced a lighter cloth with less sag, while a high density caused the cloth to fall harder and droop more noticeably. Damping controlled how quickly motion died out. With low damping, the cloth oscillated and swung for a longer time before settling, while high damping removed energy quickly and caused the cloth to reach rest much faster. Overall, these experiments showed how stiffness, mass, and energy loss each contribute differently to the cloth's motion and final shape.



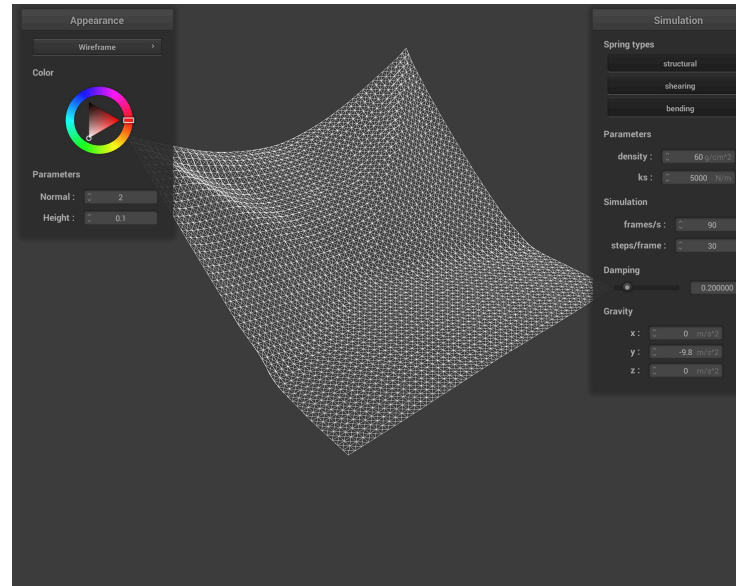
Low spring constant ($k_s = 500$): cloth is much looser and stretches more while falling.



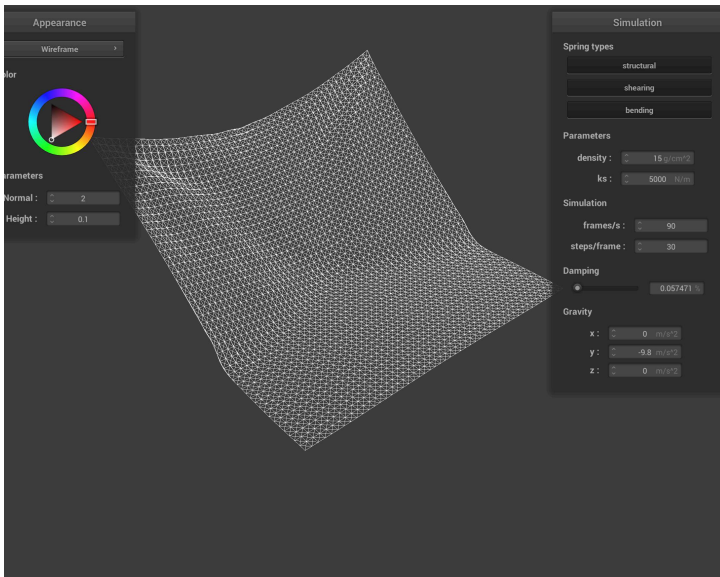
High spring constant ($k_s = 50000$): cloth is stiffer and resists stretching.



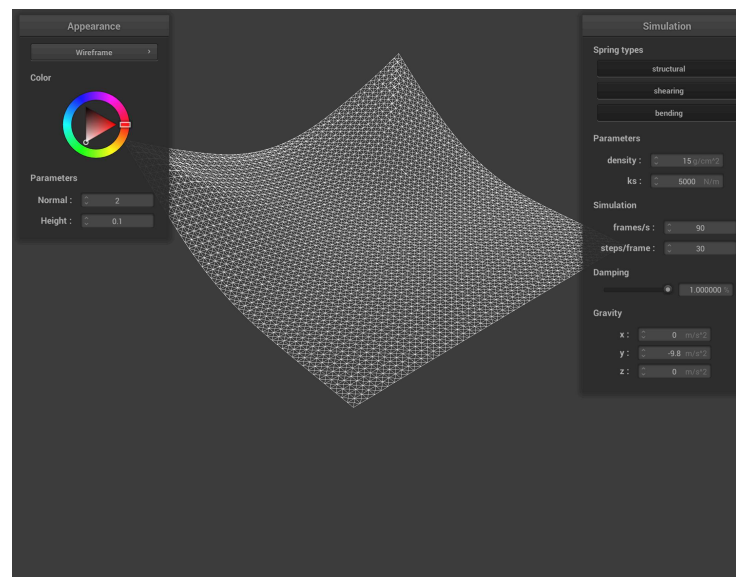
Low density ($\text{density} = 5$): lighter cloth with less sag under gravity.



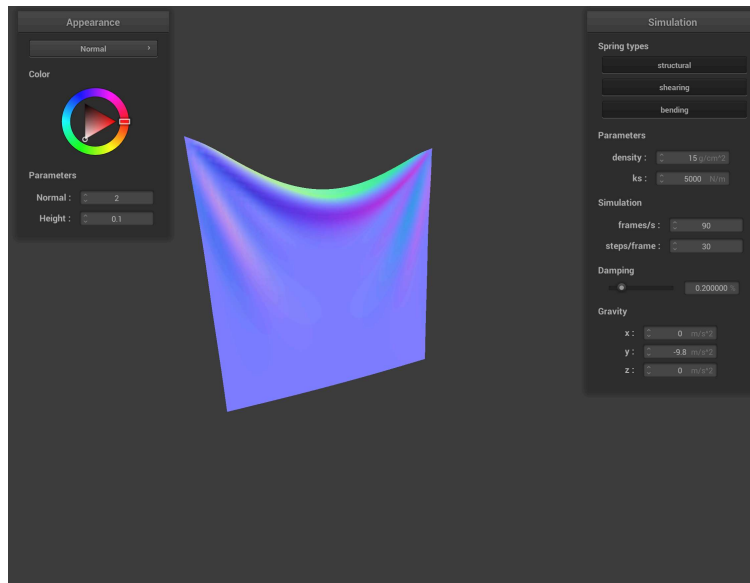
High density ($\text{density} = 60$): heavier cloth with more drooping and deformation.



Low damping ($\text{damping} = 0.05\%$): cloth keeps oscillating longer before settling.



High damping ($\text{damping} = 5\%$): cloth loses energy quickly and reaches rest faster.



Shaded resting state of pinned4.json using default parameters.

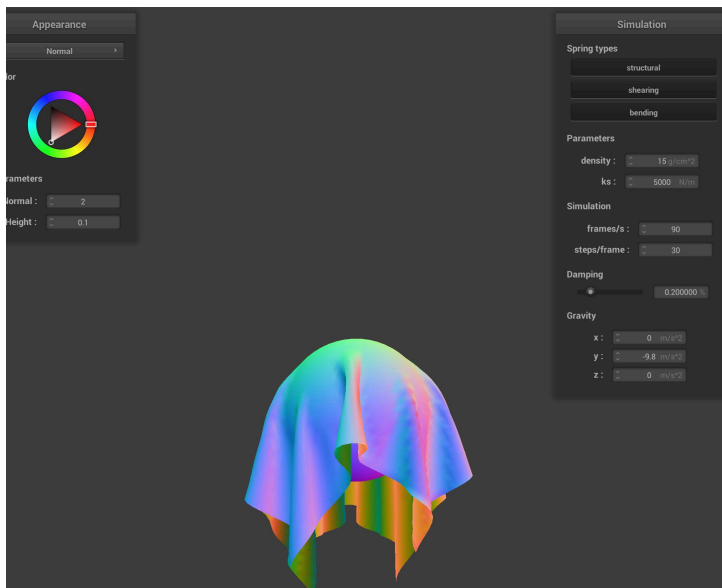
Part 3: Handling collisions with other objects

For Part 3, I implemented collision handling for both spheres and planes by updating point mass positions whenever they penetrated a collision object. For sphere collisions, I first checked whether a point mass was inside the sphere by comparing its distance from the sphere center against the radius. If it was inside, I projected the point mass outward along the direction from the sphere center to the point mass until it lay on the sphere surface. I then computed the correction vector from the point mass's previous position to this surface point and updated its position using a friction-scaled correction. This caused the cloth to be pushed back out to the sphere's surface instead of passing through it.

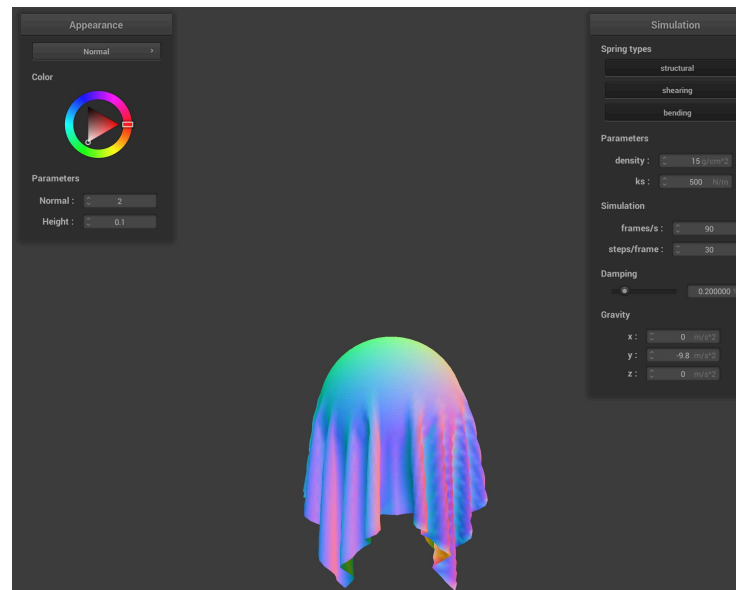
For plane collisions, I checked whether a point mass crossed from one side of the plane to the other during the last timestep by comparing the signs of the point mass's current and previous signed distances from the plane. If a crossing occurred, I computed the line-plane intersection point along the point mass's motion path. I then displaced this intersection point slightly along the plane normal using the provided surface offset so that the point mass would remain just above the plane instead of getting stuck inside it. As with the sphere, I applied the correction from the previous position and scaled it by friction. After implementing both collision methods, I

updated `cloth::simulate` so that every point mass checks for collisions against every collision object each timestep.

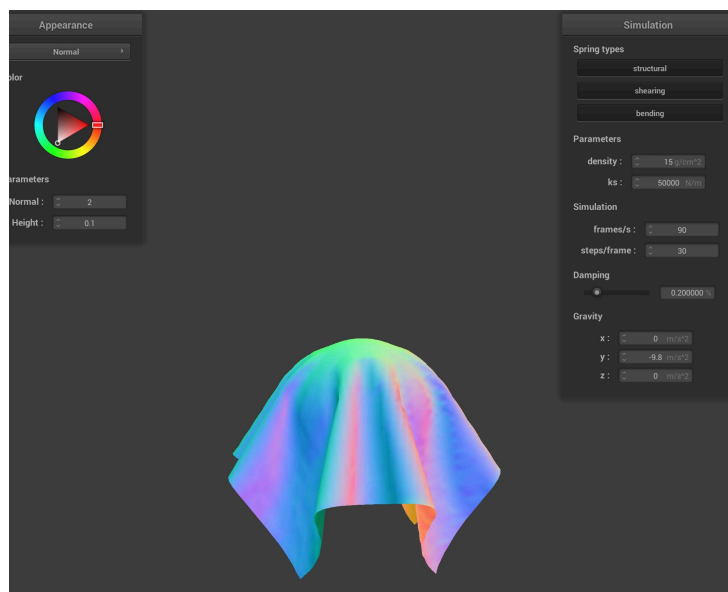
When testing with `sphere.json`, I found that the spring constant had a strong effect on the final drape of the cloth. With a low spring constant, the cloth behaved more flexibly and wrapped more closely around the sphere. With the default spring constant, the cloth still draped naturally but maintained a bit more structure. With a high spring constant, the cloth appeared much stiffer and resisted bending, so it rested on the sphere with larger, more rigid folds. In `plane.json`, the cloth fell downward and eventually came to rest flat against the plane, showing that the plane collision response correctly prevented penetration.



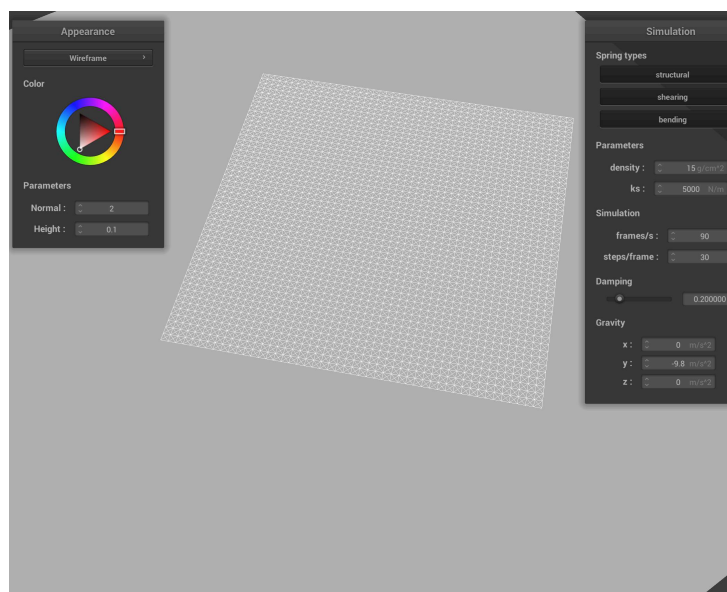
Sphere collision, default $k_s = 5000$.



Sphere collision, low $k_s = 500$: cloth drapes more loosely around the sphere.



sphere collision, high $k_s = 50000$: cloth is stiffer and folds less around the sphere.



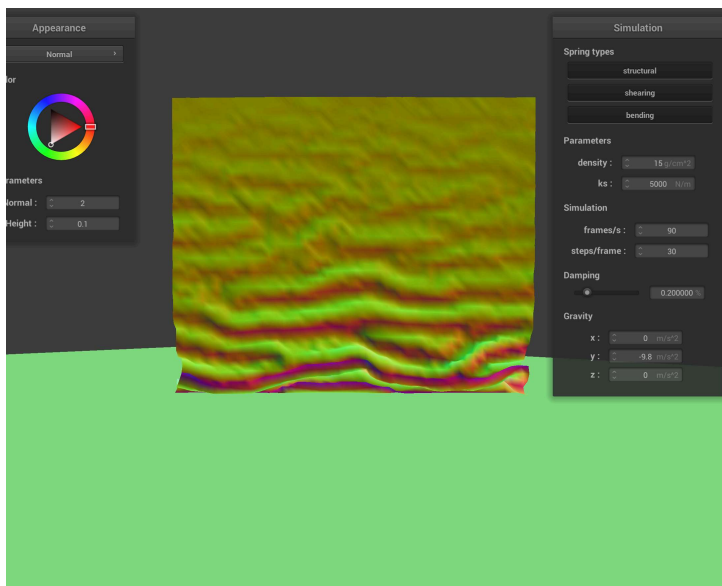
Plane collision: cloth comes to rest flat on the plane

Part 4: Handling self-collisions

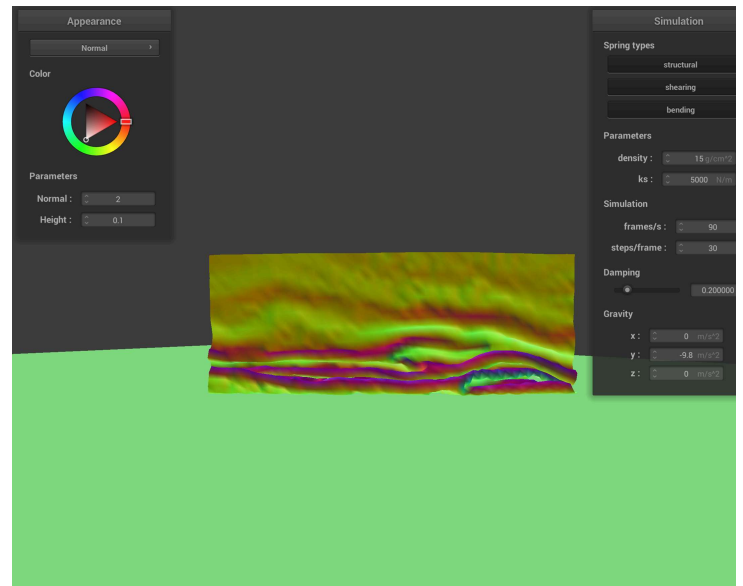
For Part 4, I implemented cloth self-collision using spatial hashing to avoid the cost of checking every pair of point masses directly. I divided space into 3D boxes whose dimensions depend on the cloth width, height, and thickness, and used a hash function to map each point mass position to a unique box key. In `build_spatial_map`, I looped through all point masses and inserted each one into the hash table bucket corresponding to its current position. This allowed each point mass to only consider nearby candidates in the same spatial region instead of comparing against the entire cloth.

In `self_collide`, I looked up the candidate point masses that shared the same hash bucket as the current point mass. For each candidate, I checked whether the two point masses were closer than $2 * \text{thickness}$. If they were, I computed a correction vector pushing the current point mass away so that the two masses would be separated by the required minimum distance. I ignored the point mass itself and averaged all valid correction vectors before applying the final correction scaled by the number of simulation steps. I also updated `simulate` so that the spatial map is rebuilt every timestep and self-collision is applied to each point mass before collisions with other primitives are resolved.

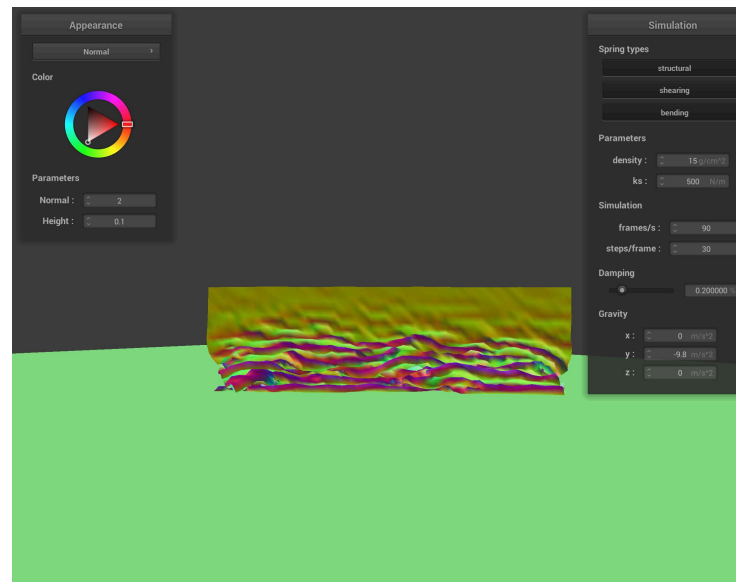
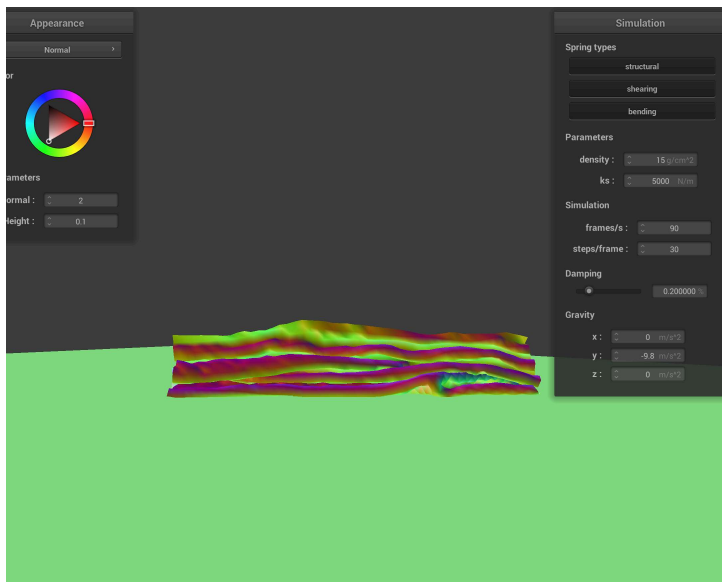
When testing with `selfCollision.json`, the cloth no longer passed through itself and instead folded onto itself as it fell toward the plane. I observed that lowering k_s made the cloth much softer, causing it to collapse into tighter folds and more wrinkles during self-collision. Increasing k_s made the cloth stiffer, so it resisted bending and formed broader, more rigid folds. Changing density affected how strongly gravity pulled the cloth downward: lower density produced a lighter cloth with gentler motion, while higher density caused the cloth to fall more aggressively and bunch up more heavily as it hit itself and the plane. Although the cloth continued to shift slightly even near rest, the main goal of preventing self-intersection was successfully achieved.



Early self-collision as the cloth begins folding onto itself.

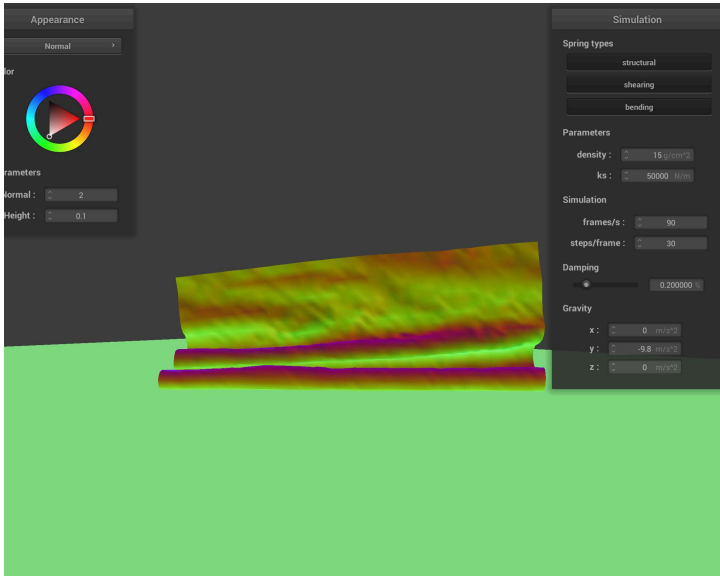


Mid-simulation folding with multiple self-contact regions.

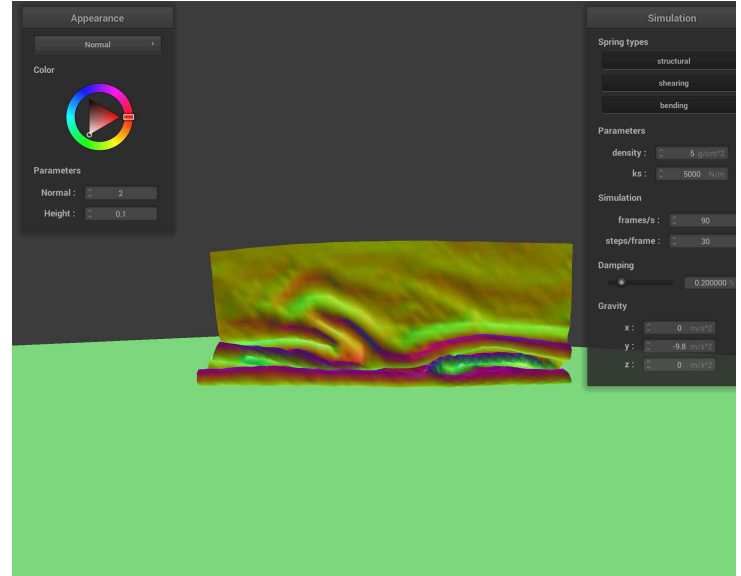


Later resting state after the cloth settles onto the plane.

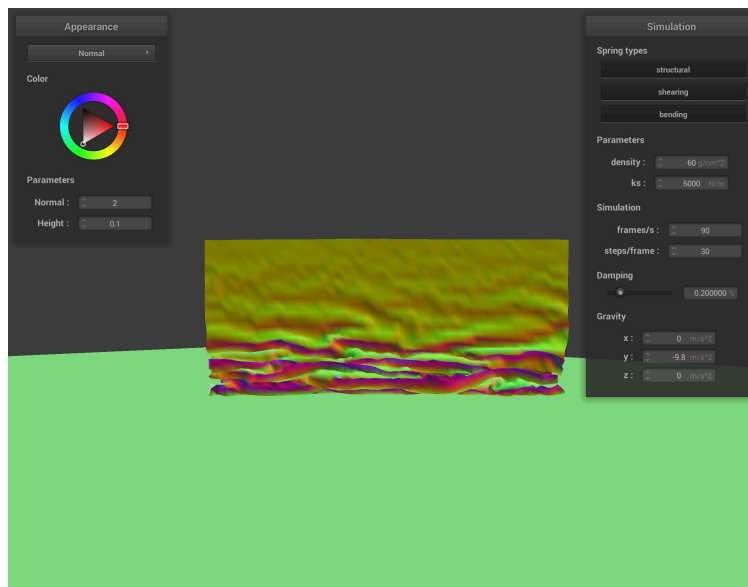
Low $k_s = 500$: softer cloth with tighter folds and more wrinkling.



High $k_s = 50000$: stiffer cloth with broader, less compressed folds.



Low density = 5: lighter cloth with gentler folding behavior.



High density = 60: heavier cloth with stronger collapse and bunching.

Part 5: Shaders

A shader program is a small program that runs on the GPU and is used to control how objects are drawn on the screen.

Instead of doing all rendering work on the CPU, shaders allow many computations to happen in parallel, which makes real-time graphics possible. In this homework, the two main types of shaders were vertex shaders and fragment shaders. The vertex shader processes each vertex of the mesh, transforming its position and passing useful information such

as normals, positions, and UV coordinates forward through varying variables. After rasterization interpolates those values across the surface of each triangle, the fragment shader runs on each fragment and computes the final color of that point on the object. In this way, the vertex shader handles geometry-related work while the fragment shader handles appearance-related work such as lighting, texture, and material effects.

The Blinn-Phong shading model is a simple lighting model that combines three pieces: ambient, diffuse, and specular lighting. The ambient term gives the object a small base amount of light so it is never completely black. The diffuse term depends on the angle between the surface normal and the light direction, so surfaces facing the light appear brighter. The specular term creates highlights and depends on how aligned the surface normal is with the halfway vector between the light direction and the view direction. Together, these three components create a surface that looks much more realistic than diffuse shading alone, since it can show both shading and shiny highlights.

For diffuse shading, I computed the lighting contribution using the dot product between the normalized surface normal and the normalized light direction. This produced a matte appearance where surfaces facing the light appeared brighter and surfaces facing away appeared darker. I then extended this into the full Blinn-Phong shader by adding ambient and specular terms. Comparing the results, the diffuse shader gave the cloth a smooth matte look, while the full Blinn-Phong shader produced a more realistic material appearance because of the visible highlights on the folds of the cloth.

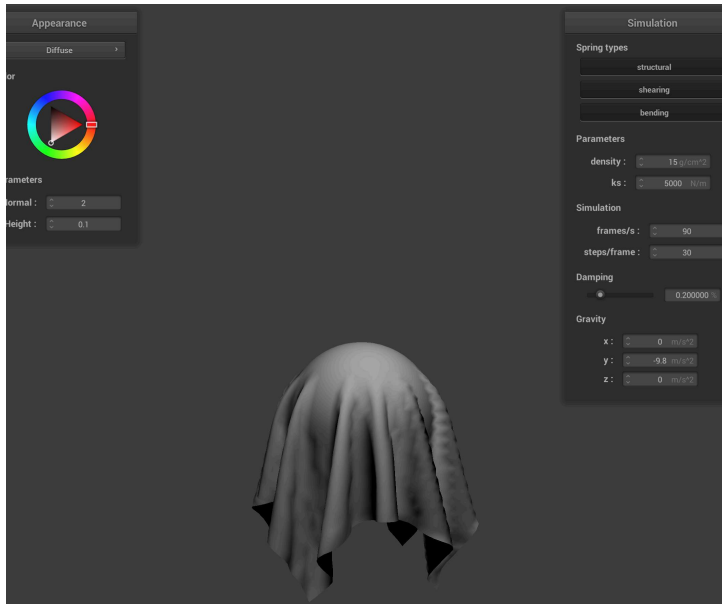
For texture mapping, I used the interpolated UV coordinates from the vertex shader to sample a custom texture image and applied that sampled color to the cloth surface. This changed the cloth from a uniform shaded object into one whose appearance was driven by the texture image itself. Because the texture is tied to UV coordinates, the image wraps across the cloth surface and bends with the simulated geometry.

For bump mapping and displacement mapping, I used a height map texture to create surface detail. In bump mapping, I did not actually move the geometry. Instead, I perturbed the

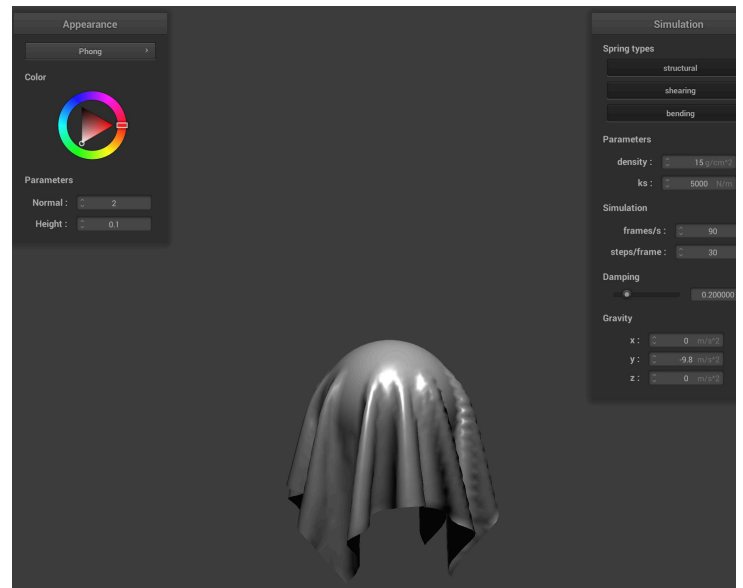
surface normals in the fragment shader using local height differences from the texture and transformed the adjusted normal using the TBN matrix. This created the illusion of bumps and wrinkles while keeping the underlying mesh unchanged. In displacement mapping, I additionally moved the vertex positions in the vertex shader along the original normal direction according to the height map. Because displacement changes the actual geometry, it produces more physically accurate silhouettes and more visible surface variation than bump mapping, especially when the mesh is sufficiently fine.

The difference between bump mapping and displacement mapping became more noticeable when comparing the cloth and sphere at different mesh resolutions. On a coarse sphere mesh, bump mapping still added visible lighting detail because it only changes normals, but displacement mapping was limited by the low geometric resolution and could not create very smooth or detailed shape changes. On a finer sphere mesh, displacement mapping became much more convincing because there were enough vertices to actually reshape the surface according to the texture. This makes displacement mapping more geometry-dependent, while bump mapping is cheaper and still effective even on lower-resolution meshes.

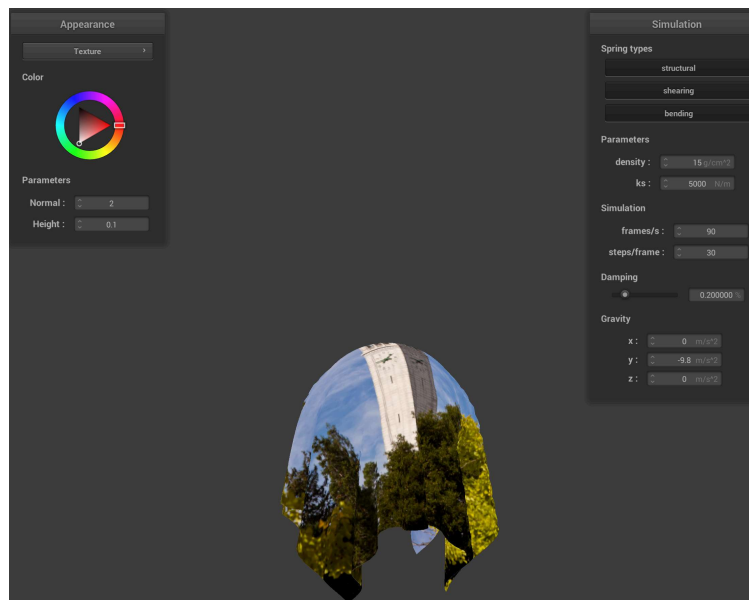
For the mirror shader, I computed the outgoing view direction from the camera position to the fragment position, reflected it across the surface normal, and then sampled the cubemap in that reflected direction. This made both the cloth and sphere behave like reflective surfaces that capture the surrounding environment. The result looked especially striking on the folds of the cloth, since different parts of the surface reflected different parts of the environment map.



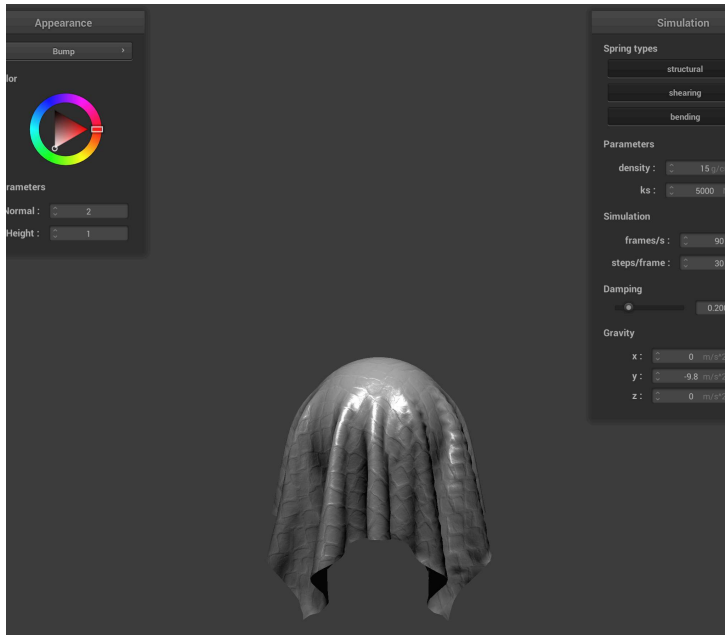
Diffuse shader output on the cloth.



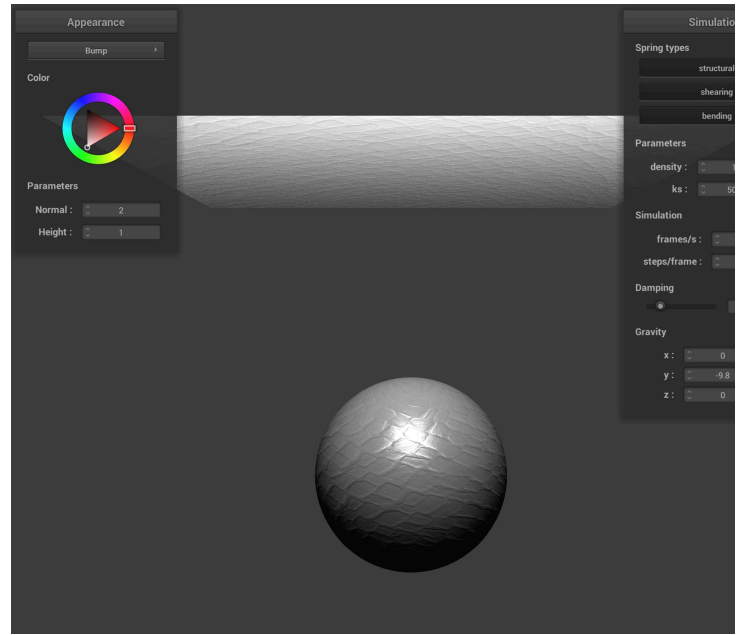
Complete Blinn-Phong shading output.



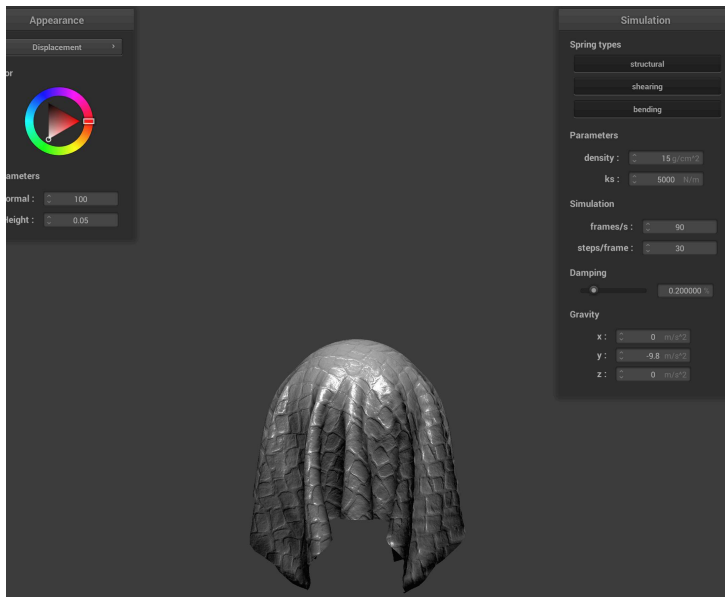
Texture mapping using a custom texture on the cloth.



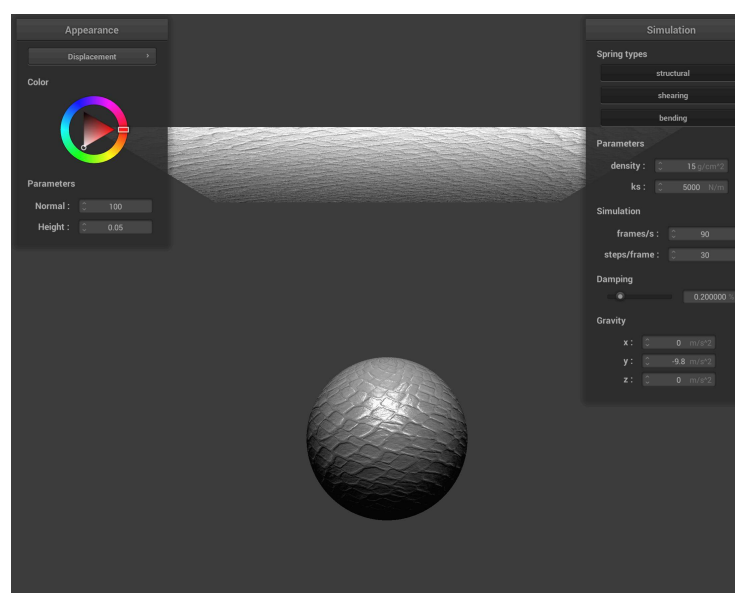
Bump mapping on the cloth using a non-default height texture.



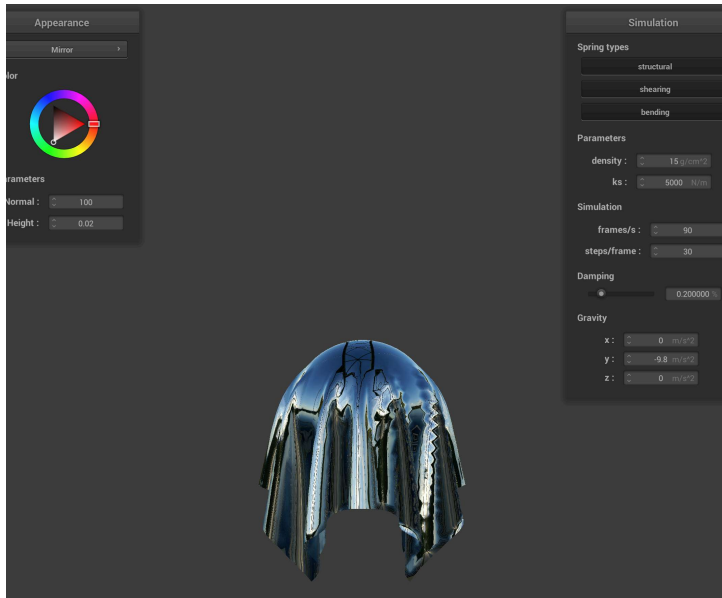
Bump mapping on the sphere using the same height texture.



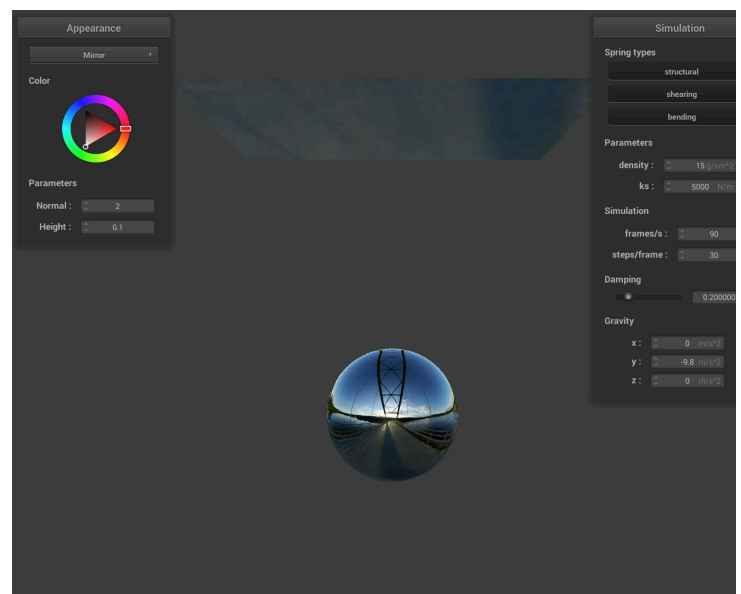
Displacement mapping on the cloth.



Displacement mapping on the sphere.



Mirror shader on the cloth with environment-mapped reflections.



Mirror shader on the sphere with environment-mapped reflections.

Bump vs. displacement comparison. Bump mapping and displacement mapping both use a height texture to add detail, but they do so in different ways. Bump mapping only changes the surface normals used for lighting, so it gives the illusion of detail without changing the actual mesh geometry. Displacement mapping changes the vertex positions themselves, so it produces real geometric variation and more accurate silhouettes. Because of this, displacement mapping looks more convincing on high-resolution meshes, while bump mapping is lighter-weight and still effective even when the mesh is coarse.

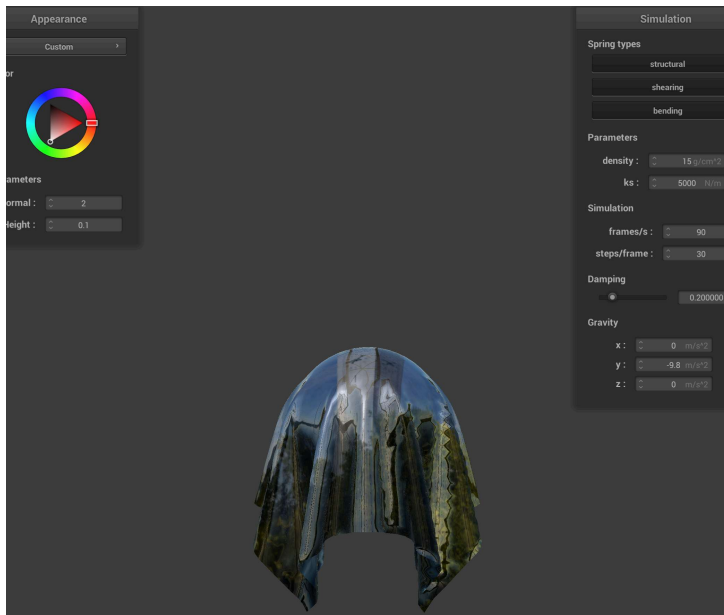
Coarse vs. fine sphere comparison. On the coarse sphere, bump mapping still showed lighting detail because it only depended on altered normals, but displacement mapping was limited by the small number of vertices and appeared less smooth. On the finer sphere, displacement mapping became much more detailed because the additional vertices allowed the texture to reshape the geometry more accurately. This made the difference between the two approaches much clearer at high resolution.

Part 6: Extra Credit - Additional cloth simulation features!

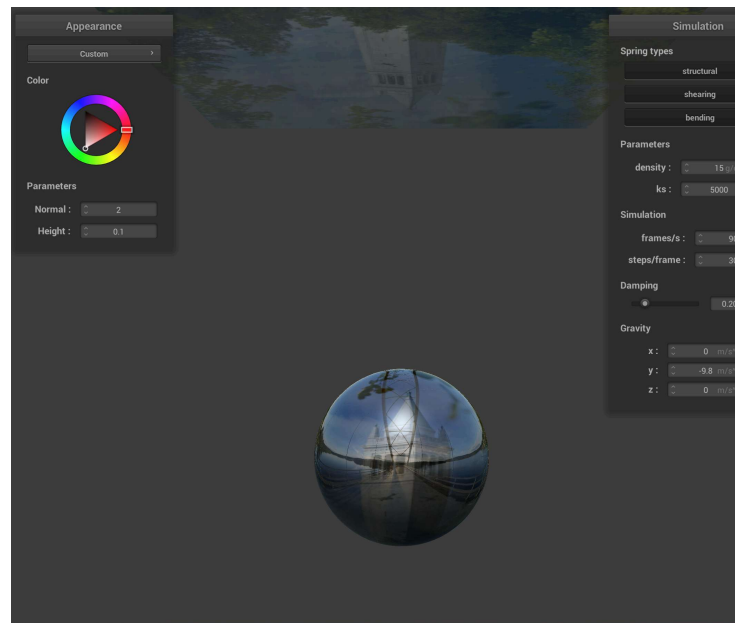
For the extra credit shader, I created a custom reflective material by extending the mirror shader. Instead of using only the cubemap reflection color, I blended the environment reflection with the sampled texture color and combined that result with Blinn-Phong lighting. I also added a Fresnel-style weighting so that the reflections become stronger at grazing angles, which makes the material look more dynamic on folds and curved surfaces.

This shader combines several ideas from the earlier parts of the assignment. It uses texture mapping to give the cloth and sphere a base surface pattern, uses ambient, diffuse, and specular lighting to preserve local shading detail, and then mixes in environment-mapped reflections for a more metallic appearance. The Fresnel-style effect helps the reflection respond to the viewing angle, so the surface looks less flat and more visually interesting as the camera moves.

Compared to the standard mirror shader, the custom shader produces a more stylized result because it keeps the surface color visible instead of replacing it entirely with the cubemap reflection. This gave the cloth and sphere a tinted reflective appearance that felt more expressive while still reacting to light and view direction in a believable way.



Custom shader on the cloth combining texture mapping, Blinn-Phong shading, and environment reflections.



Custom shader on the sphere with tinted reflection and a Fresnel-style effect.